

AI PERSONAL STYLING • CAPSTONE PROJECT REPORT

FitCheck *AI*

Your Personal AI Fashion Stylist

A wardrobe and styling web application featuring a digital closet, context-aware outfit generation, selfie-based color-season analysis, travel packing, and AI virtual try-on — built with React and FastAPI and deployed on Vercel.

Student Aiperi Kudaibergenova

Instructor Abdulkirim Oncu

Date July 2026

Abstract

FitCheck AI is an AI personal-styling and wardrobe assistant that helps style-conscious users digitize their wardrobe and receive context-aware styling guidance. The product combines six connected experiences: a digital closet, an outfit generator that reacts to weather, calendar and formality, a selfie-based color-season analysis engine, a travel packing planner, an AI virtual try-on shopping surface, and a personal dashboard. It is implemented as a React single-page application backed by a FastAPI REST service, uses SQLAlchemy for persistence and Pillow for deterministic image colour analysis, and is packaged for one-click deployment to Vercel — serving the static frontend and a Python serverless API from a single origin.

This report presents the motivation and problem statement, the functional and non-functional requirements, the system architecture and data model, the API surface, the design system, detailed feature walkthroughs, implementation and deployment notes, testing, security considerations, and future work. A complete appendix reproduces the core source code of the system for reference.

Contents

1. Introduction
2. Objectives & Scope
3. Background & Market Context
4. Requirements Analysis
5. System Overview & Features
6. System Architecture
7. Data Model & Database Design
8. API Design & Endpoint Reference
9. Technology Stack
10. Design System
11. Feature Deep-Dives
12. Implementation Notes
13. Deployment on Vercel
14. Testing & Quality Assurance
15. Security Considerations
16. Challenges & Solutions

17. Future Work

18. Conclusion

References

Appendix A — Source Code

1. Introduction

Getting dressed is a small decision people make every single day, yet it carries a surprising amount of friction. A good outfit has to satisfy several constraints at once: it must suit the weather, be appropriate for the day's occasions, flatter the wearer, and feel like an expression of personal taste. Repeating this decision every morning is tiring, and the cost of getting it wrong — feeling under- or over-dressed — is real.

At the same time, most people dramatically underuse the wardrobe they already own. A commonly cited pattern is that people wear a small fraction of their clothes most of the time while the rest sits forgotten at the back of the closet. The problem is not usually a lack of clothing; it is a lack of visibility and combination ideas. People simply cannot see everything they own at once, and they run out of fresh ways to combine it.

FitCheck AI addresses this by turning a physical wardrobe into a structured, searchable digital closet and then layering AI styling on top of it. Once a user has photographed and tagged their clothes, the application can suggest complete outfits tuned to the day's context, recommend colours that flatter the user, assemble a packing list for a trip, and surface new pieces to buy that fit the user's palette. Everything is delivered through a calm, premium interface inspired by quiet-luxury fashion, so the experience feels like guidance from a personal stylist rather than data entry into a spreadsheet.

Consider a representative morning. A user opens the app, and the dashboard already shows a suggested outfit for the day that accounts for the forecast and the two meetings on the calendar. The user likes it, saves it, and moves on in seconds. Later that week, planning a weekend trip, the same user enters a destination and dates and receives a packing list built entirely from clothes they own. This is the loop FitCheck AI is designed to make effortless.

2. Objectives & Scope

2.1 Objectives

- **Digitize the wardrobe:** let users photograph and tag clothing into a searchable digital closet.
- **Automate outfit decisions:** generate outfit suggestions grounded in the user's own items and daily context.
- **Personalize colour:** diagnose a user's colour season from a selfie and produce a tailored palette.
- **Support real scenarios:** help with travel packing and with shopping in flattering colours.
- **Deliver a premium experience:** a coherent luxury-fashion design system across every screen.

- **Ship it:** package the full stack for straightforward cloud deployment on Vercel.

2.2 In Scope

The project delivers a working full-stack web application: account registration and login, wardrobe management with image upload, context-aware outfit generation, colour-season analysis, travel packing, a curated shopping surface with virtual try-on, a profile and dashboard, and a light/dark theme. It also delivers the deployment configuration required to run the application on Vercel.

2.3 Out of Scope

Real-time integration with live retailer inventories, native mobile applications, automatic garment recognition from photos, and social/community features are considered future work and are discussed in Section 17. The colour-analysis engine is deliberately implemented as transparent, deterministic colour science rather than a trained machine-learning model, though its interface is designed so that such a model can be substituted later.

3. Background & Market Context

The product sits at the intersection of luxury fashion and consumer AI. Its reference points in feel are brands such as The Row, COS, Massimo Dutti and Net-a-Porter — restrained, editorial, and premium. The 'quiet luxury' movement, which favours understated quality over conspicuous branding, directly informs the visual language: a neutral canvas that never competes with the clothing, a single refined accent colour, and generous white space.

Existing wardrobe and outfit applications tend to feel utilitarian — closer to inventory management than to styling. FitCheck AI's differentiation is to treat the experience as a considered, aspirational one, targeted specifically at style-conscious users who value aesthetics. This positioning shapes not only the interface but also the copy, the demo wardrobes, and the imagery.

The colour-analysis feature draws on the established twelve-season colour-analysis system, a framework widely used by professional stylists. The system classifies a person's natural colouring along perceptual dimensions — undertone (cool versus warm), depth (light versus deep) and clarity (muted versus clear) — and maps the result to a season (for example, Deep Autumn or Cool Summer) with an associated flattering palette. FitCheck AI implements this as explainable, reproducible colour science, which keeps results predictable and debuggable and avoids the opacity of a black-box model.

4. Requirements Analysis

4.1 Functional Requirements

ID	Requirement
FR-1	Users can register an account and log in with email and password.
FR-2	Users can add clothing items with a photo, category, colour and season.
FR-3	Users can browse, filter and delete items in their digital closet.
FR-4	The system generates outfit suggestions from the user's own wardrobe.
FR-5	Outfit generation accounts for weather, calendar events and formality.
FR-6	Users can save favourite outfits and retrieve a suggestion for 'today'.
FR-7	Users can upload a selfie and receive a colour-season diagnosis and palette.
FR-8	The system recommends products and colours based on the user's palette.
FR-9	Users can generate a travel packing list from a destination, dates and plans.
FR-10	Users can preview items using an AI virtual try-on.
FR-11	Users can view and edit a profile and upload an avatar.
FR-12	Users can switch between a light and a dark theme.

4.2 Non-Functional Requirements

- **Usability:** the interface must feel premium and calm, with a consistent design system and minimal-intentional motion.
- **Performance:** pages should load quickly; the frontend is a compiled static bundle and API responses are lightweight JSON.
- **Portability:** the backend must run with zero configuration locally (SQLite, local file storage) and scale to production services (PostgreSQL, S3) purely through environment variables.

- **Security:** passwords must never be stored in plaintext; access-control and CORS behaviour must be configurable per environment.
- **Maintainability:** colour values, typography and spacing must be centralized as design tokens rather than scattered literals.
- **Deployability:** the whole stack must deploy to a single Vercel project from a single Git push.

4.3 Representative User Stories

- As a busy professional, I want a ready outfit each morning so that I don't waste time deciding.
- As a frequent traveller, I want a packing list built from my own clothes so that I never over-pack or forget essentials.
- As a shopper, I want to know which colours flatter me so that I stop buying pieces I never wear.
- As a design-conscious user, I want the app itself to feel beautiful so that using it is a pleasure.

5. System Overview & Features

FitCheck AI is organized into six connected experiences, unified by a single account and wardrobe. Each experience maps to a page in the frontend and one or more routers in the backend API.

Digital Wardrobe. Upload garments with photos; tag by category, colour and season; browse and filter the closet. Backed by the closet API (create, upload, list, delete).

Outfit Generator. Produces outfit suggestions from the user's wardrobe, tuned by weather, calendar events and formality; supports saving outfits and a 'today' pick.

Colour-Season Analysis. Analyses a selfie to classify the user into one of twelve colour seasons and returns a named palette plus colours to avoid.

Smart Shopping. Curated shopping items with an AI virtual try-on so users can preview pieces before buying.

Travel Packing. Generates a complete packing list from a destination, dates and planned activities, aware of the trip's climate.

Profile & Dashboard. Account and profile management, avatar upload, saved outfits, wardrobe gap insights and daily AI suggestions.

6. System Architecture

The system follows a classic decoupled client-server architecture with three logical layers: a presentation layer (the React SPA), an application layer (the FastAPI service), and a data layer (the relational database and object storage).

6.1 Presentation Layer — React SPA

The client is a single-page application built with React 19 and React Router. It is organized into pages (Landing, Auth, Dashboard, Wardrobe, Generator, Travel, Shopping, Profile) and shared components (an application shell with navigation, an item modal, a theme toggle and a toast notifier). A small, typed API client wraps the browser fetch API, injects the authentication token, and centralizes error handling. Application state such as authentication status and the active theme is kept in React state and mirrored to localStorage so it survives reloads.

6.2 Application Layer — FastAPI

The backend is a FastAPI application composed of independent routers, one per domain area: closet, outfits, colour, travel, shopping, suggestions, weather, calendar, auth and health. Each router declares its routes with typed request and response models validated by Pydantic, which also generates interactive OpenAPI documentation automatically. A middleware layer applies an optional IP allowlist and configurable CORS. The signature colour-analysis algorithm lives in a dedicated module and is pure Python over Pillow.

6.3 Data Layer — Database & Storage

Persistence uses the SQLAlchemy 2 ORM. In development the database is a zero-configuration SQLite file; in production the same code targets PostgreSQL simply by changing the DATABASE_URL environment variable. User-uploaded images are handled by a storage abstraction with two interchangeable backends — the local filesystem (served through a static mount) or AWS S3 with presigned URLs — chosen automatically from configuration. The OpenAI API powers the shopping virtual try-on.

6.4 Request Lifecycle

A typical authenticated request flows as follows: the React client attaches a bearer token to the request and calls the API; FastAPI matches the route, runs dependencies (including a database session and the current-user lookup), validates the request body against a Pydantic schema, executes the handler, and returns a validated JSON response. On Vercel, the request first hits the platform's edge, which serves static assets directly and forwards /api requests to the Python serverless function running the FastAPI application.

6.5 Deployment Topology

On Vercel the two tiers are served from one origin. The compiled React build is hosted as static files, and every `/api` request is routed to a Python serverless function that mounts the FastAPI application. Because both are served from the same domain, no cross-origin configuration is required — the frontend calls the API at a relative `/api` base.

7. Data Model & Database Design

The relational schema is defined with SQLAlchemy models. The principal entities are the user and the clothing item, complemented by saved outfits, calendar events, and the stored colour-analysis result. All user-owned records carry a foreign key to their owner so that data is cleanly partitioned per account.

Entity	Purpose	Key fields
User	An account holder.	id, email, password_hash, name, city, avatar
Clothing item	A garment in the digital closet.	id, user_id, name, category, colour, season, image
Saved outfit	An outfit the user kept.	id, user_id, item references, context/label
Calendar event	A dated event that informs formality.	id, user_id, title, date, type
Colour result	The stored season diagnosis.	id, user_id, season, palette, avoid

Passwords are never stored in plaintext; only a bcrypt hash of the password is persisted. Images are stored by an opaque key, and the application resolves that key to a browser-loadable URL through the storage abstraction, so switching from local files to S3 requires no schema change.

8. API Design & Endpoint Reference

The API is REST-style and returns JSON. Routes are grouped by router; representative endpoints are listed below. Interactive documentation is available at `/docs` when the server is running.

Method & Path	Description
GET <code>/health</code>	Liveness check.
POST <code>/auth/register</code>	Create an account; returns a bearer token.
POST <code>/auth/login</code>	Authenticate; returns a bearer token.

GET /auth/me	Return the current user's profile.
PUT /auth/me	Update the current user's profile.
POST /auth/avatar	Upload a profile avatar image.
GET /closet	List the user's clothing items.
POST /closet	Create a clothing item.
POST /closet/upload	Create an item from an uploaded image.
DELETE /closet/{id}	Delete a clothing item.
POST /outfits/generate	Generate an outfit for a given context.
GET /outfits/today	Return today's suggested outfit.
POST /outfits/save	Save an outfit.
GET /outfits/saved	List saved outfits.
POST /outfits/try-on	Virtual try-on for a generated outfit.
POST /color/analyze	Analyse an uploaded selfie into a colour season.
GET /color/result	Return the stored colour-analysis result.
GET /color/recommendations	Palette-based recommendations.
GET /color/seasons	Reference data for all twelve seasons.
POST /travel/pack	Generate a packing list.
GET /shopping/items	List curated shopping items.
POST /shopping/try-on	AI virtual try-on for a shopping item.
GET /suggestions/today	Daily AI suggestion.
GET /suggestions/gaps	Wardrobe gap analysis.
GET /weather/current	Current weather used by the generator.
GET /calendar/events	List calendar events.

9. Technology Stack

Layer	Technologies	Role
Frontend	React 19, React Router 7, Tailwind CSS 3, framer-motion, lucide-react	SPA UI, routing, styling, motion, icons
Backend	FastAPI, Pydantic 2, bcrypt, python-multipart	REST API, validation, auth, uploads

Data	SQLAlchemy 2, SQLite / PostgreSQL (psycopg2)	ORM and persistence (dev / prod)
Imaging & AI	Pillow, OpenAI API	Colour-season analysis, virtual try-on
Storage	Local filesystem / AWS S3 (boto3)	User image storage
Hosting	Vercel (static build + Python serverless)	Frontend + API deployment

The stack was chosen for developer velocity and portability: FastAPI provides typed, self-documenting endpoints with minimal boilerplate; SQLAlchemy abstracts the database so the same code runs on SQLite and PostgreSQL; and React with Tailwind allows a bespoke, design-led interface without a heavy component framework.

10. Design System

The interface follows a documented luxury-fashion design language so that the product feels premium and consistent across every screen.

- **Aesthetic:** quiet luxury — a soft ivory canvas, a single gold accent, generous white space, and large photography; the clothing and the gold detail are the heroes.
- **Typography:** Playfair Display, a high-contrast editorial serif, for headlines and the wordmark; Poppins, a geometric sans, for UI and body text; tabular numerals for data such as prices and sizes.
- **Colour:** semantic CSS-variable tokens (canvas, card, field, line, fg, muted, gold) drive both a light (ivory + gold) and a dark (espresso + gold) theme through a single app-wide toggle, so every component adapts automatically.
- **Layout:** near-flat corners, hairline borders, an eight-pixel spacing system, product grids, and an asymmetric editorial composition on the marketing surface.
- **Motion:** minimal-intentional — gentle fades and image reveals, underlines that grow from the left, and a maximum hover zoom just above one; never bouncy or spring-like.

A signature screen — the colour-season reveal — is treated as an editorial spread rather than a dashboard card: the season name is set large in the display serif, the palette is shown as flush rectangular colour fields rather than rounded chips, and a single restrained 'Shop your palette' link invites the next step.

11. Feature Deep-Dives

11.1 Colour-Season Analysis

The selfie-to-season pipeline is the headline feature and is fully deterministic, using Pillow only — there is no face-recognition model or external vision API. The algorithm proceeds in five stages: (1) it decodes and downscales the selfie for speed; (2) it extracts the dominant skin tone using a centre-weighted skin-pixel mask, falling back to colour quantization when too few skin pixels are detected; (3) it projects that tone onto three perceptual axes — undertone, depth and clarity — derived from the CIELAB colour space; (4) it classifies the result into the twelve-season system by nearest archetype; and (5) it returns the season, a curated hex palette with named roles, and a list of colours to avoid.

A deliberate design choice is that the function's output shape is stable and documented. This means a trained vision model can later be slotted in behind the same interface without changing any consumer of the result — the rest of the application is insulated from the implementation. The full implementation is reproduced in Appendix A.

11.2 Context-Aware Outfit Generation

The generator only ever combines items the user actually owns, which keeps every suggestion actionable. It adapts to real context by reading the current weather, the day's calendar events, and a formality signal, and biases its selection accordingly — for example, warmer layers on a cold day, or more formal pieces when the calendar shows a meeting. Users can save the outfits they like and request a ready 'today' pick, and can preview a generated look with virtual try-on before committing to it.

11.3 Travel Packing

The travel planner converts a destination, a date range and a set of planned activities into a complete, climate-aware packing list assembled from the user's wardrobe. Rather than a generic checklist, the output reflects what the user owns and what the trip actually requires, reducing both over-packing and forgotten essentials.

11.4 Smart Shopping & Virtual Try-On

The shopping surface presents curated pieces, ideally aligned to the user's colour palette, and uses the OpenAI API to render a virtual try-on so that a user can visualize a piece before buying. This closes the loop from analysis (what suits me) to acquisition (what to buy next).

12. Implementation Notes

- **API client:** a thin fetch wrapper injects the bearer token, centralizes error handling, and reads its base URL from the `REACT_APP_API_URL` environment variable so the same build works locally (pointing at localhost) and in production (pointing at a same-origin `/api`).
- **Authentication:** registration and login return a bearer token; passwords are hashed with `bcrypt` on the server. Protected routes on the frontend redirect unauthenticated users to the login screen, and the backend exposes endpoints to fetch and update the current profile.
- **Storage abstraction:** callers deal only with opaque keys and a `public_url(key)` helper, so the rest of the code is agnostic to whether files live on the local disk or in S3; `boto3` is imported lazily so the dependency is only needed when S3 is actually configured.
- **Colour engine:** implemented as a standalone, dependency-light module over `Pillow` with a stable result contract, making it independently testable and replaceable.
- **Configuration:** the database URL, allowed CORS origins, an optional IP allowlist, the S3 settings and the OpenAI key are all environment-driven, keeping secrets and environment differences out of the codebase.
- **Theming:** colours are defined once as CSS-variable tokens and wired into Tailwind, so the light and dark themes are produced from the same class names without duplicated styles.

13. Deployment on Vercel

The repository is configured for a single-project Vercel deployment that serves both tiers from one domain. The configuration files are reproduced in Appendix A.

- **vercel.json:** declares two builds — a static build that compiles the React app and a Python build that bundles the serverless API — and routes that send `/api` requests to the function, serve static assets, and fall back to the SPA's `index.html` for client-side routing.
- **api/index.py:** the Python serverless entrypoint. It places the FastAPI backend on the import path, supplies serverless-safe defaults, and mounts the application under `/api` so the backend's own routes match unchanged.
- **requirements.txt (root):** declares the Python dependencies Vercel installs for the API function.
- **Same-origin API:** the production frontend is built with `REACT_APP_API_URL` set to `/api`, so it calls the API on the same domain with no CORS configuration.

Production checklist.

- **Set DATABASE_URL** to a managed PostgreSQL instance (for example Neon, Supabase, or Vercel Postgres). Serverless functions are stateless, so the bundled SQLite fallback is intended for demos only and resets between cold starts.
- **Set S3_BUCKET and AWS credentials** so uploaded selfies and avatars persist beyond a single request; the local uploads directory is read-only and ephemeral on Vercel.
- **Set OPENAI_API_KEY** to enable the virtual try-on, and ALLOWED_ORIGINS if any client is served from a different domain.

With these in place, pushing to the connected Git repository triggers Vercel to build the frontend, bundle the API, and publish a live URL.

14. Testing & Quality Assurance

- **Build verification:** the production React build compiles cleanly, and the serverless entrypoint imports and serves the FastAPI application, with the health endpoint returning a 200 status through the deployment routing.
- **API routing checks:** representative endpoints (health, colour seasons, application root) were exercised through the same /api-prefixed path the browser uses, confirming the mount and routing behave as intended.
- **Manual end-to-end testing:** the core flows — sign up, add wardrobe items, generate an outfit, run colour analysis, and build a packing list — were exercised against the API.
- **Design QA:** screens were reviewed against the documented design system to flag any legacy theme colours, off-scale typography, or non-token values, keeping the interface consistent.

15. Security Considerations

Several security-relevant decisions are built into the system, and some limitations are noted honestly for future hardening.

- **Password storage:** passwords are hashed with bcrypt and never stored or logged in plaintext.
- **Configurable access control:** the API supports a configurable CORS origin list and an optional IP allowlist that can lock the service to specific machines for private deployments.
- **Secret management:** all credentials (database, S3, OpenAI) are supplied through environment variables rather than being committed to the repository.
- **Known limitation:** the current bearer token is a simple, unsigned token derived from the user's email rather than a signed JSON Web Token. This is adequate for a demonstration but should be replaced with a signed, expiring token before any public deployment, and is listed as future work.

16. Challenges & Solutions

Challenge	Problem	Solution
Explainable colour analysis	Needed reliable, debuggable results without a heavy ML dependency.	Built a deterministic CIELAB pipeline in Pillow with a stable output contract that a model can later replace.
Light/dark theming	A legacy hardcoded green theme made two-mode support difficult.	Migrated every screen to semantic CSS-variable tokens so both themes adapt automatically.
Serverless statelessness	Functions have ephemeral, read-only filesystems and no persistent local DB.	Externalized all state through environment-driven PostgreSQL and an S3 storage backend.
Monorepo deployment	A React app in a subfolder plus a Python API confused the build's working directory.	Adopted Vercel's path-explicit static-build and Python builds so each tier builds in its own folder.
Design consistency	Keeping a premium, coherent feel across many screens.	Enforced a single gold accent, a strict type scale, and reusable shell and card components.

17. Future Work

- **Trained vision model:** replace the deterministic analyzer behind the same interface for richer, photograph-robust colour and style analysis.
- **Automatic garment tagging:** detect the category and colour of uploaded items automatically to remove manual data entry.
- **Signed authentication:** move from the simple bearer token to a signed, expiring JSON Web Token with refresh handling.
- **Social features:** shareable lookbooks and community feedback on outfits.
- **Native mobile app:** a camera-first companion for capturing the wardrobe and selfies on the go.
- **Retailer integrations:** purchase recommended pieces directly within the shopping flow using live inventory.

18. Conclusion

FitCheck AI demonstrates a complete, deployable AI product: a premium React frontend, a well-structured FastAPI backend, explainable colour science, and a same-origin Vercel deployment

that ships both tiers together. It solves a genuine everyday problem — turning a full closet into confident, effortless styling — while remaining honest about its engineering, with clear paths to extend it toward trained models, automation, stronger authentication and mobile. The system meets its functional objectives and provides a solid, maintainable foundation for the future work described above.

References

- [1] FastAPI documentation — <https://fastapi.tiangolo.com>
- [2] React documentation — <https://react.dev>
- [3] Tailwind CSS documentation — <https://tailwindcss.com>
- [4] SQLAlchemy documentation — <https://www.sqlalchemy.org>
- [5] Pillow (PIL) documentation — <https://python-pillow.org>
- [6] Vercel documentation — <https://vercel.com/docs>
- [7] OpenAI API reference — <https://platform.openai.com/docs>
- [8] The twelve-season colour-analysis system (industry reference).
- [9] CIELAB colour space — International Commission on Illumination (CIE).

Appendix A — Source Code

This appendix reproduces the core source code of FitCheck AI for reference. Listings are grouped by area. Line numbers are shown at the left of each listing. Complete source, including the remaining pages and components, is available in the project repository.

A.1 Deployment Configuration

Vercel configuration `vercel.json`

```
1 {
2   "$schema": "https://openapi.vercel.sh/vercel.json",
3   "version": 2,
4   "builds": [
5     {
6       "src": "style_front/package.json",
7       "use": "@vercel/static-build",
8       "config": { "distDir": "build" }
9     },
10    {
11      "src": "api/index.py",
12      "use": "@vercel/python"
13    }
14  ],
15  "routes": [
16    { "src": "/api/(.*)", "dest": "/api/index.py" },
17    { "handle": "filesystem" },
18    { "src": "/(.*)", "dest": "/index.html" }
19  ]
20 }
```

Serverless API entrypoint `api/index.py`

```
1 """
2 Vercel Python (serverless) entrypoint for the FitCheck AI backend.
3
4 Vercel routes every ``/api/*`` request to this file — see ``vercel.json``. The
5 real FastAPI application lives in ``../style_back/main.py``. Here we:
6
7 1. put ``style_back`` on ``sys.path`` so ``from main import app`` resolves,
8 2. supply serverless-safe env defaults, and
9 3. expose a real FastAPI ``app`` that mounts the backend under ``/api``.
10
11 Vercel preserves the original request path through the rewrite, so a request to
12 ``/api/closet`` reaches this app as ``/api/closet``; the mount forwards it to
the
13 backend as ``/closet`` and its routers match unchanged. Exposing a genuine
14 FastAPI instance (rather than a bare ASGI callable) guarantees Vercel's Python
15 runtime detects it as an ASGI app.
16
17 — Serverless caveats (see DEPLOYMENT.md) —
18 Vercel functions are stateless with an ephemeral, read-only filesystem (only
19 ``/tmp`` is writable). For anything beyond a throwaway demo you MUST set:
20
21 • ``DATABASE_URL`` → a managed Postgres (Neon, Supabase, Vercel Postgres).
22   The ``/tmp`` SQLite fallback below resets on cold
start.
23 • ``S3_BUCKET`` (+ AWS creds) → so uploaded selfies/avatars persist. The
local
```

```

24         ``uploads/`` directory is read-only/ephemeral on
Vercel.
25     """
26
27     import os
28     import sys
29     from pathlib import Path
30
31     # — 1. Make the FastAPI backend importable —————
32     BACKEND_DIR = Path(__file__).resolve().parent.parent / "style_back"
33     sys.path.insert(0, str(BACKEND_DIR))
34
35     # — 2. Serverless-safe defaults (real values come from Vercel env vars)
36     os.environ.setdefault("DATABASE_URL", "sqlite:///tmp/fitcheck.db")
37
38     from fastapi import FastAPI # noqa: E402
39     from main import app as backend_app # noqa: E402 (import after sys.path
tweak)
40
41     # — 3. Mount the backend under /api so its own routes match unchanged
42
43     app = FastAPI(title="FitCheck AI (Vercel)")
44     app.mount("/api", backend_app)

```

Python dependencies (serverless) `requirements.txt`

```

1 # Vercel Python serverless dependencies for the FitCheck AI backend.
2 # Mirrors style_back/requirements.txt (Vercel reads this file at the repo root
3 # to build the /api serverless function).
4 fastapi==0.115.6
5 python-multipart==0.0.18
6 pydantic==2.10.3
7 pydantic-settings==2.6.1
8 sqlalchemy>=2.0.0
9 psycopg2-binary>=2.9.0
10 bcrypt>=4.0.0
11 python-dotenv>=1.0.0
12 Pillow>=10.0.0
13 boto3>=1.34.0
14 openai>=1.40.0

```

A.2 Backend — Application Core

FastAPI application `style_back/main.py`

```

1 import os
2 from pathlib import Path
3 from fastapi import FastAPI, Request
4 from fastapi.responses import JSONResponse
5 from fastapi.middleware.cors import CORSMiddleware
6 from fastapi.staticfiles import StaticFiles
7
8 from app.database import engine
9 from app.models import Base
10 from app.routers import health, closet, outfits, auth, suggestions, calendar,
weather, travel, shopping, color
11
12 Base.metadata.create_all(bind=engine)
13
14 UPLOADS_DIR = Path(__file__).parent / "uploads"
15 UPLOADS_DIR.mkdir(exist_ok=True)

```

```

16
17 # — Access control —————
18 # Optional IP allowlist (opt-in). When ALLOWED_IPS is set (e.g. for a private
19 # LAN deployment), only those client IPs may reach the API. When UNSET (the
20 # default – normal production), no IP restriction is applied. Set it in .env to
21 # lock the app to specific machines.
22 ALLOWED_IPS = {
23     ip.strip() for ip in os.getenv("ALLOWED_IPS", "").split(",") if ip.strip()
24 }
25 # CORS: comma-separated frontend origins. Defaults to localhost for dev; set
26 # ALLOWED_ORIGINS to your deployed frontend URL(s) in production.
27 ALLOWED_ORIGINS = [o.strip() for o in os.getenv(
28     "ALLOWED_ORIGINS",
29     "http://localhost:3000,http://127.0.0.1:3000",
30 ).split(",") if o.strip()]
31
32 app = FastAPI(
33     title="ASANUR API",
34     description="Backend for the ASANUR AI styling and wardrobe assistant",
35     version="0.2.0",
36 )
37
38
39 @app.middleware("http")
40 async def restrict_client_ip(request: Request, call_next):
41     # No restriction unless an allowlist is configured.
42     if ALLOWED_IPS:
43         client = request.client.host if request.client else None
44         if client not in ALLOWED_IPS:
45             return JSONResponse(status_code=403, content={"detail":
"Forbidden"})
46     return await call_next(request)
47
48
49 app.add_middleware(
50     CORSMiddleware,
51     allow_origins=ALLOWED_ORIGINS,
52     allow_credentials=True,
53     allow_methods=["*"],
54     allow_headers=["*"],
55 )
56
57 app.include_router(health.router)
58 app.include_router(closet.router)
59 app.include_router(outfits.router)
60 app.include_router(auth.router)
61 app.include_router(suggestions.router)
62 app.include_router(calendar.router)
63 app.include_router(weather.router)
64 app.include_router(travel.router)
65 app.include_router(shopping.router)
66 app.include_router(color.router)
67
68 app.mount("/uploads", StaticFiles(directory=str(UPLOADS_DIR)), name="uploads")
69
70
71 @app.get("/")
72 def root():
73     return {"message": "ASANUR API", "docs": "/docs"}

```

Database configuration `style_back/app/database.py`

```

1 import os
2 from dotenv import load_dotenv
3 from sqlalchemy import create_engine

```

```

4  from sqlalchemy.orm import sessionmaker, DeclarativeBase
5
6  load_dotenv()
7
8  DATABASE_URL = os.getenv("DATABASE_URL", "")
9
10 if DATABASE_URL.startswith("sqlite"):
11     engine = create_engine(DATABASE_URL, connect_args={"check_same_thread":
False}, pool_pre_ping=True)
12 else:
13     engine = create_engine(DATABASE_URL, pool_pre_ping=True)
14 SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
15
16
17 class Base(DeclarativeBase):
18     pass
19
20
21 def get_db():
22     db = SessionLocal()
23     try:
24         yield db
25     finally:
26         db.close()

```

ORM models `style_back/app/models.py`

```

1  from sqlalchemy import Column, Integer, String, Text, DateTime, Float
2  from sqlalchemy.sql import func
3  from app.database import Base
4
5
6  class UserModel(Base):
7     __tablename__ = "users"
8
9     id = Column(Integer, primary_key=True, index=True)
10    email = Column(String(255), unique=True, index=True, nullable=False)
11    password_hash = Column(String(255), nullable=False)
12    name = Column(String(255), nullable=True)
13    city = Column(String(255), nullable=True)
14    created_at = Column(DateTime(timezone=True), server_default=func.now())
15    height_cm = Column(Float, nullable=True)
16    weight_kg = Column(Float, nullable=True)
17    avatar_url = Column(Text, nullable=True)
18
19
20 class ClosetItemModel(Base):
21     __tablename__ = "closet_items"
22
23     id = Column(Integer, primary_key=True, index=True)
24     user_id = Column(Integer, nullable=True, index=True)
25     name = Column(String(255), nullable=False)
26     category = Column(String(50), nullable=False)
27     image_url = Column(Text, nullable=True)
28     color = Column(String(50), nullable=True)
29     formality = Column(String(50), nullable=True)
30     formality_value = Column(Integer, nullable=True)
31     created_at = Column(DateTime(timezone=True), server_default=func.now())
32
33
34 class CalendarEventModel(Base):
35     __tablename__ = "calendar_events"
36
37     id = Column(String(36), primary_key=True)
38     user_id = Column(Integer, nullable=True, index=True)

```

```

39     title = Column(String(255), nullable=False)
40     start = Column(String(30), nullable=False)
41     end_time = Column(String(30), nullable=False)
42     formality = Column(String(50), nullable=True)
43     created_at = Column(DateTime(timezone=True), server_default=func.now())
44
45
46     class SavedOutfitModel(Base):
47         __tablename__ = "saved_outfits"
48
49         id = Column(String(36), primary_key=True)
50         user_id = Column(Integer, nullable=True, index=True)
51         context = Column(String(100), nullable=False)
52         items_json = Column(Text, nullable=False)
53         explanation = Column(Text, nullable=False)
54         saved_at = Column(DateTime(timezone=True), server_default=func.now())
55
56
57     class ColorAnalysisModel(Base):
58         __tablename__ = "color_analyses"
59
60         id = Column(Integer, primary_key=True, index=True)
61         user_id = Column(Integer, nullable=True, index=True)
62         selfie_key = Column(Text, nullable=True)          # storage key (local path
or S3 object key)
63         season = Column(String(40), nullable=False)      # e.g. "Deep Winter"
64         undertone = Column(String(20), nullable=True)    # warm | cool | neutral
65         skin_hex = Column(String(7), nullable=True)      # detected dominant skin
tone
66         palette_json = Column(Text, nullable=False)      #
[{"name","hex","role"}, ...]
67         avoid_json = Column(Text, nullable=True)         # [{"hex", ...}]
68         description = Column(Text, nullable=True)
69         created_at = Column(DateTime(timezone=True), server_default=func.now())
70
71
72     class ShoppingItemModel(Base):
73         __tablename__ = "shopping_items"
74
75         id = Column(Integer, primary_key=True, index=True)
76         name = Column(String(255), nullable=False)
77         brand = Column(String(255), nullable=True)
78         category = Column(String(50), nullable=False)    #
Top/Bottom/Outerwear/Shoes/Accessory
79         price = Column(Float, nullable=True)
80         color = Column(String(50), nullable=True)
81         description = Column(Text, nullable=True)
82         image_url = Column(Text, nullable=True)
83         tags = Column(String(255), nullable=True)        # comma-separated e.g.
"sport,casual"
84         store_url = Column(Text, nullable=True)          # link to the product page on the
store website

```

Storage abstraction (local / S3) `style_back/app/storage.py`

```

1  """
2  Storage abstraction for user-uploaded images (selfies, avatars, etc.).
3
4  Two interchangeable backends:
5
6  • LocalStorage - writes to the existing ``uploads/`` directory and serves
7                  files through FastAPI's StaticFiles mount at ``/uploads``.
8                  This is the default and requires zero configuration, so the
9                  app runs end-to-end on any machine.
10

```

```

11     • S3Storage      - uploads objects to an AWS S3 bucket and hands back short-
lived
12                        presigned GET URLs for viewing. Activated automatically
when
13                        the ``S3_BUCKET`` environment variable is present.
14     ``boto3`` is
15                        imported lazily so the dependency is only required when S3
is
16                        actually used.
17     The active backend is chosen once by :func:`get_storage`. Callers deal only
with
18     opaque ``key`` strings (e.g. ``"selfies/ab12.jpg"``) plus a ``public_url(key)``
19     that resolves to something a browser can load – identical surface in both
modes.
20     """
21
22     from __future__ import annotations
23
24     import os
25     import uuid
26     from pathlib import Path
27     from typing import Optional
28
29     # uploads/ lives at the backend root (same directory main.py mounts as
StaticFiles)
30     UPLOADS_DIR = Path(__file__).parent.parent / "uploads"
31     UPLOADS_DIR.mkdir(exist_ok=True)
32
33     _EXT_BY_CONTENT_TYPE = {
34         "image/jpeg": ".jpg",
35         "image/jpg": ".jpg",
36         "image/png": ".png",
37         "image/webp": ".webp",
38     }
39
40
41     def _safe_ext(filename: Optional[str], content_type: Optional[str]) -> str:
42         """Pick a safe image extension from the filename, falling back to content-
type."""
43         if filename:
44             ext = Path(filename).suffix.lower()
45             if ext in (".jpg", ".jpeg", ".png", ".webp"):
46                 return ".jpg" if ext == ".jpeg" else ext
47             return _EXT_BY_CONTENT_TYPE.get((content_type or "").lower(), ".jpg")
48
49
50     def make_key(prefix: str, filename: Optional[str] = None, content_type:
Optional[str] = None) -> str:
51         """Build a collision-free storage key like ``selfies/3f2a....jpg``."""
52         ext = _safe_ext(filename, content_type)
53         return f"{prefix.strip('/')}/{uuid.uuid4().hex}{ext}"
54
55
56     class StorageBackend:
57         """Interface implemented by every storage backend."""
58
59         name: str = "base"
60
61         def save(self, data: bytes, key: str, content_type: str = "image/jpeg") ->
str:
62             raise NotImplementedError
63
64         def public_url(self, key: str) -> str:
65             """Return a browser-loadable URL for a stored key."""
66             raise NotImplementedError
67
68         def delete(self, key: str) -> None:

```

```

69         raise NotImplementedError
70
71
72     class LocalStorage(StorageBackend):
73         """Writes files under ``uploads/`` and serves them via the ``/uploads``
mount."""
74
75         name = "local"
76
77         def __init__(self, base_dir: Path = UPLOADS_DIR):
78             self.base_dir = base_dir
79             self.base_dir.mkdir(exist_ok=True)
80
81         def _path(self, key: str) -> Path:
82             # Keys may contain a prefix dir (e.g. "selfies/x.jpg"); create it.
83             dest = self.base_dir / key
84             dest.parent.mkdir(parents=True, exist_ok=True)
85             return dest
86
87         def save(self, data: bytes, key: str, content_type: str = "image/jpeg") ->
str:
88             self._path(key).write_bytes(data)
89             return key
90
91         def public_url(self, key: str) -> str:
92             # Relative URL; the frontend prepends REACT_APP_API_URL.
93             return f"/uploads/{key}"
94
95         def delete(self, key: str) -> None:
96             try:
97                 self._path(key).unlink(missing_ok=True)
98             except OSError:
99                 pass
100
101
102     class S3Storage(StorageBackend):
103         """Uploads to S3 and returns presigned GET URLs. boto3 imported lazily."""
104
105         name = "s3"
106
107         def __init__(self):
108             self.bucket = os.environ["S3_BUCKET"]
109             self.region = os.environ.get("AWS_REGION", "us-east-1")
110             self.url_ttl = int(os.environ.get("S3_URL_TTL_SECONDS", "3600"))
111             try:
112                 import boto3 # lazy: only needed in S3 mode
113             except ImportError as exc: # pragma: no cover - depends on env
114                 raise RuntimeError(
115                     "S3_BUCKET is set but boto3 is not installed. "
116                     "Run `pip install boto3` or unset S3_BUCKET to use local
storage."
117                 ) from exc
118             # Credentials resolve from the standard AWS chain (env, profile, IAM
role).
119             self._client = boto3.client("s3", region_name=self.region)
120
121         def save(self, data: bytes, key: str, content_type: str = "image/jpeg") ->
str:
122             self._client.put_object(
123                 Bucket=self.bucket,
124                 Key=key,
125                 Body=data,
126                 ContentType=content_type,
127             )
128             return key
129
130         def public_url(self, key: str) -> str:
131             return self._client.generate_presigned_url(

```

```

132         "get_object",
133         Params={"Bucket": self.bucket, "Key": key},
134         ExpiresIn=self.url_ttl,
135     )
136
137     def delete(self, key: str) -> None:
138         try:
139             self._client.delete_object(Bucket=self.bucket, Key=key)
140         except Exception:
141             pass
142
143
144     _storage: Optional[StorageBackend] = None
145
146
147     def get_storage() -> StorageBackend:
148         """Return the process-wide storage backend (S3 when configured, else
149         local)."""
150         global _storage
151         if _storage is None:
152             _storage = S3Storage() if os.environ.get("S3_BUCKET") else
153             LocalStorage()
154         return _storage

```

Authentication helpers `style_back/app/auth_utils.py`

```

1  from typing import Optional
2  from fastapi import Header, Depends
3  from sqlalchemy.orm import Session
4  from app.database import get_db
5  from app.models import UserModel
6
7
8  def get_current_user(
9      authorization: Optional[str] = Header(None),
10     db: Session = Depends(get_db),
11 ) -> Optional[UserModel]:
12     if not authorization or not authorization.startswith("Bearer token-"):
13         return None
14     email = authorization[len("Bearer token-"):]
15     return db.query(UserModel).filter(UserModel.email == email).first()
16
17
18  def get_current_user_id(
19     authorization: Optional[str] = Header(None),
20     db: Session = Depends(get_db),
21 ) -> Optional[int]:
22     if not authorization or not authorization.startswith("Bearer token-"):
23         return None
24     email = authorization[len("Bearer token-"):]
25     user = db.query(UserModel).filter(UserModel.email == email).first()
26     return user.id if user else None

```

A.3 Backend — Signature Feature: Colour-Season Analysis

Selfie → colour-season pipeline `style_back/app/color_analysis.py`

```

1  """
2  Selfie → color-season analysis. Pillow only, fully deterministic.
3
4  Pipeline:
5      1. Decode + downscale the selfie.

```



```

6     2. Extract the dominant *skin* tone (center-weighted skin-pixel mask, with a
7       quantization fallback when too few skin pixels are found).
8     3. Project that tone onto three perceptual axes – undertone (cool→warm),
9       depth (light→deep), clarity (muted→clear) – derived from CIELAB.
10    4. Classify into the 12-season color system by nearest archetype.
11    5. Return the season, a curated hex palette (with named roles) and an
12       "avoid" list.
13
14    It is honest color science, not a face-recognition model: no opencv/dlib, so it
15    installs and runs anywhere Pillow does. The result shape is stable, so a true
16    vision model can later be slotted in behind :func:`analyze_selfie` unchanged.
17    """
18
19    from __future__ import annotations
20
21    import io
22    from typing import Optional
23
24    from PIL import Image
25
26    from app.color_map import RGB, rgb_to_hex, rgb_to_lab
27
28    MAX_DIM = 220 # downscale target – plenty for color statistics, fast to
process
29
30
31    # — 12-season archetypes on (undertone, depth, clarity)  $\in [-1, 1]^3$  —————
32    # undertone: -1 cool → +1 warm
33    # depth:     -1 light → +1 deep
34    # clarity:   -1 muted → +1 clear
35    _SEASONS: dict[str, dict] = {
36        "Light Spring": {
37            "vector": (0.5, -0.8, 0.2),
38            "undertone": "warm",
39            "description": "Warm and delicate. Light, fresh tones with a golden
glow look effortless on you.",
40            "palette": [
41                ("Peach", "#FFCBA4", "signature"), ("Coral", "#FF7F50", "pop"),
42                ("Warm Mint", "#B8E0D2", "fresh"), ("Light Camel", "#D9B99B",
"neutral"),
43                ("Buttercup", "#F3E37C", "accent"), ("Aqua", "#7FDD8D", "cool-
pop"),
44                ("Ivory", "#FFFFFF", "base"), ("Salmon Pink", "#FA8072", "soft"),
45            ],
46            "avoid": ["#000000", "#36454F", "#800020"],
47        },
48        "Warm Spring": {
49            "vector": (0.9, 0.0, 0.4),
50            "undertone": "warm",
51            "description": "Golden and radiant. Rich, warm hues with yellow
undertones bring out your warmth.",
52            "palette": [
53                ("Golden Amber", "#D97706", "signature"), ("Warm Coral", "#FF6F50",
"pop"),
54                ("Olive", "#708238", "neutral"), ("Turquoise", "#40C4B0", "cool-
pop"),
55                ("Camel", "#C19A6B", "base"), ("Tomato", "#E14B36", "accent"),
56                ("Cream", "#FFFDD0", "soft"), ("Moss", "#8A9A5B", "earthy"),
57            ],
58            "avoid": ["#000000", "#B0E0E6", "#915F6D"],
59        },
60        "Bright Spring": {
61            "vector": (0.4, 0.0, 0.9),
62            "undertone": "warm",
63            "description": "Clear and vivid. High-energy, saturated colors with
warmth suit your bright contrast.",
64            "palette": [

```

```

65         ("Bright Coral", "#FF5349", "signature"), ("Emerald", "#1FB47A",
"pop"),
66         ("Cobalt", "#1565C0", "cool-pop"), ("Hot Pink", "#EC4899",
"accent"),
67         ("Golden Yellow", "#EAB308", "warm-pop"), ("Ivory", "#FFFFFF0",
"base"),
68         ("Bright Navy", "#1F2A6B", "neutral"), ("Apple Green", "#8DC63F",
"fresh"),
69     ],
70     "avoid": ["#9CAF88", "#B38B6D", "#915F6D"],
71 },
72     "Light Summer": {
73         "vector": (-0.5, -0.8, -0.2),
74         "undertone": "cool",
75         "description": "Cool and soft. Gentle, light tones with a blue base
keep your look luminous.",
76         "palette": [
77             ("Powder Blue", "#B0E0E6", "signature"), ("Soft Rose", "#DE9C9C",
"pop"),
78             ("Lavender", "#C7B8E0", "accent"), ("Dove Grey", "#C9CBD0",
"neutral"),
79             ("Mint", "#B8E0D2", "fresh"), ("Periwinkle", "#9DB4D6", "cool"),
80             ("Soft White", "#F7F5F1", "base"), ("Cool Pink", "#E6A8C6",
"soft"),
81         ],
82         "avoid": ["#000000", "#D97706", "#B7410E"],
83     },
84     "Cool Summer": {
85         "vector": (-0.9, 0.0, -0.2),
86         "undertone": "cool",
87         "description": "Pure cool tones. Blue-based colors with medium softness
flatter you most.",
88         "palette": [
89             ("Soft Fuchsia", "#C154A0", "signature"), ("Cool Blue", "#3B6CA8",
"pop"),
90             ("Spruce", "#3F6F6F", "neutral"), ("Raspberry", "#B03A6E",
"accent"),
91             ("Slate Blue", "#6A7BA2", "base"), ("Cool Mint", "#9FD8C8",
"fresh"),
92             ("Soft White", "#F2F2F0", "soft"), ("Plum", "#8E4585", "deep"),
93         ],
94         "avoid": ["#D97706", "#C19A6B", "#EAB308"],
95     },
96     "Soft Summer": {
97         "vector": (-0.4, 0.0, -0.8),
98         "undertone": "cool",
99         "description": "Muted and cool. Hazy, dusty tones with a soft blue base
bring quiet elegance.",
100         "palette": [
101             ("Dusty Rose", "#C76B7E", "signature"), ("Sage", "#9CAF88",
"neutral"),
102             ("Mauve", "#915F6D", "pop"), ("Slate", "#708090", "base"),
103             ("Soft Teal", "#5F9EA0", "cool"), ("Cocoa", "#6F5B52", "deep"),
104             ("Pewter", "#A8A9AD", "soft"), ("Soft Plum", "#7E5C73", "accent"),
105         ],
106         "avoid": ["#FF5349", "#EAB308", "#000000"],
107     },
108     "Soft Autumn": {
109         "vector": (0.4, 0.0, -0.8),
110         "undertone": "warm",
111         "description": "Muted and warm. Earthy, low-saturation tones with a
golden base look harmonious.",
112         "palette": [
113             ("Terracotta", "#C66B3D", "signature"), ("Sage", "#9CAF88",
"neutral"),
114             ("Camel", "#C19A6B", "base"), ("Soft Olive", "#8A8B5C", "earthy"),
115             ("Salmon", "#E08A6E", "pop"), ("Teal", "#4F8A8B", "cool"),

```

```

116         ("Warm Beige", "#D9C2A6", "soft"), ("Brick", "#9C5B3F", "deep"),
117     ],
118     "avoid": ["#FF5349", "#000000", "#B0E0E6"],
119 },
120 "Warm Autumn": {
121     "vector": (0.9, 0.3, -0.2),
122     "undertone": "warm",
123     "description": "Rich and golden. Deep, spicy earth tones with strong
warmth are your power colors.",
124     "palette": [
125         ("Rust", "#B7410E", "signature"), ("Mustard", "#C9A227", "pop"),
126         ("Olive", "#708238", "neutral"), ("Pumpkin", "#D2691E", "accent"),
127         ("Chocolate", "#5C4033", "base"), ("Forest", "#0B6623", "cool"),
128         ("Cream", "#F5EAD0", "soft"), ("Teal", "#13767F", "deep"),
129     ],
130     "avoid": ["#B0E0E6", "#EC4899", "#000000"],
131 },
132 "Deep Autumn": {
133     "vector": (0.5, 0.9, 0.0),
134     "undertone": "warm",
135     "description": "Deep and warm. Dark, rich tones with golden depth give
you striking presence.",
136     "palette": [
137         ("Burnt Orange", "#A84B2A", "signature"), ("Deep Teal", "#0E5A5A",
"pop"),
138         ("Espresso", "#3B2A20", "base"), ("Olive", "#5B6133", "neutral"),
139         ("Brick Red", "#8B2E1E", "accent"), ("Mustard", "#B8860B", "warm-
pop"),
140         ("Cream", "#EFE3C8", "soft"), ("Forest", "#14452F", "deep"),
141     ],
142     "avoid": ["#B0E0E6", "#F7F5F1", "#E6A8C6"],
143 },
144 "Deep Winter": {
145     "vector": (-0.5, 0.9, 0.2),
146     "undertone": "cool",
147     "description": "Deep and cool. Bold, dark tones with a cool base create
dramatic contrast on you.",
148     "palette": [
149         ("True Black", "#000000", "base"), ("Sapphire", "#0F52BA",
"signature"),
150         ("Burgundy", "#800020", "pop"), ("Emerald", "#046307", "accent"),
151         ("Icy White", "#F4F8FB", "soft"), ("Deep Plum", "#4E2A4E", "deep"),
152         ("Pine", "#13433C", "neutral"), ("Crimson", "#9E1B32", "warm-pop"),
153     ],
154     "avoid": ["#D9B99B", "#C19A6B", "#9CAF88"],
155 },
156 "Cool Winter": {
157     "vector": (-0.9, 0.3, 0.2),
158     "undertone": "cool",
159     "description": "Pure and cool. Icy, blue-based colors with crisp
contrast suit you best.",
160     "palette": [
161         ("Royal Blue", "#1E3A8A", "signature"), ("Fuchsia", "#C2185B",
"pop"),
162         ("Icy Pink", "#F2C6D6", "soft"), ("Emerald", "#0B7A52", "accent"),
163         ("Charcoal", "#36454F", "neutral"), ("Cool Ruby", "#9B1B40",
"deep"),
164         ("Pure White", "#FFFFFF", "base"), ("Cobalt", "#0047AB", "cool-
pop"),
165     ],
166     "avoid": ["#D97706", "#C19A6B", "#EAB308"],
167 },
168 "Bright Winter": {
169     "vector": (-0.4, 0.2, 0.9),
170     "undertone": "cool",
171     "description": "Clear and cool. Vivid, saturated colors with icy
contrast make you shine.",

```

```

172         "palette": [
173             ("Hot Pink", "#E91E63", "signature"), ("Electric Blue", "#1565C0",
"pop"),
174             ("Emerald", "#00875A", "accent"), ("True Red", "#D81E2C", "warm-
pop"),
175             ("Pure White", "#FFFFFF", "base"), ("Black", "#000000", "neutral"),
176             ("Icy Violet", "#B39DDB", "soft"), ("Lemon", "#F4E04D", "bright"),
177         ],
178         "avoid": ["#C19A6B", "#9CAF88", "#B38B6D"],
179     },
180 }
181
182
183 # — skin-tone extraction —————
184 def _is_skin(r: int, g: int, b: int) -> bool:
185     """Kovac et al. RGB skin-detection rule (uniform daylight)."""
186     mx, mn = max(r, g, b), min(r, g, b)
187     return (
188         r > 95 and g > 40 and b > 20
189         and (mx - mn) > 15
190         and abs(r - g) > 15
191         and r > g and r > b
192     )
193
194
195 def _dominant_skin_rgb(img: Image.Image) -> RGB:
196     """Average skin-colored pixels in the central region; quantize as
fallback."""
197     img = img.convert("RGB")
198     img.thumbnail((MAX_DIM, MAX_DIM))
199     w, h = img.size
200
201     # Center crop (faces tend to sit centrally in a selfie) reduces background
bias.
202     cx0, cy0 = int(w * 0.2), int(h * 0.15)
203     cx1, cy1 = int(w * 0.8), int(h * 0.85)
204     center = img.crop((cx0, cy0, max(cx1, cx0 + 1), max(cy1, cy0 + 1)))
205
206     pixels = list(center.getdata())
207     skin = [(r, g, b) for (r, g, b) in pixels if _is_skin(r, g, b)]
208
209     if len(skin) >= 25:
210         n = len(skin)
211         return (
212             sum(p[0] for p in skin) // n,
213             sum(p[1] for p in skin) // n,
214             sum(p[2] for p in skin) // n,
215         )
216
217     # Fallback: quantize to a small palette and pick the most skin-like /
common.
218     qimg = center.convert("RGB").quantize(colors=6, method=Image.MEDIANCUT)
219     palette = qimg.getpalette() or []
220     counts = qimg.getcolors() or [] # list of (count, palette_index)
221     counts.sort(reverse=True)
222     best_rgb: Optional[RGB] = None
223     for _, idx in counts:
224         rgb = (palette[idx * 3], palette[idx * 3 + 1], palette[idx * 3 + 2])
225         if _is_skin(*rgb):
226             best_rgb = rgb
227         break
228     if best_rgb is None and counts:
229         idx = counts[0][1]
230         best_rgb = (palette[idx * 3], palette[idx * 3 + 1], palette[idx * 3 +
231         2])
232     return best_rgb or (200, 160, 130)

```

```

233
234 # — season classification —————
235 def _axes_from_skin(rgb: RGB) -> tuple[float, float, float]:
236     """Map a skin RGB to (undertone, depth, clarity), each clamped to [-1,
237 1]."""
238     L, a, b = rgb_to_lab(rgb)
239     chroma = (a * a + b * b) ** 0.5
240
241     def clamp(x: float) -> float:
242         return max(-1.0, min(1.0, x))
243
244     # warmth: yellow (b) dominance over red (a). ~6 = neutral.
245     undertone = clamp((b - a - 6.0) / 8.0)
246     # depth: darker skin → deeper coloring. L≈93 light, L≈43 deep.
247     depth = clamp((68.0 - L) / 25.0)
248     # clarity: higher chroma → clearer; lower → muted. ~22 = neutral.
249     clarity = clamp((chroma - 22.0) / 12.0)
250     return undertone, depth, clarity
251
252 def _classify(axes: tuple[float, float, float]) -> str:
253     u, d, c = axes
254     best_name, best_dist = "Soft Autumn", float("inf")
255     for name, spec in _SEASONS.items():
256         tu, td, tc = spec["vector"]
257         dist = (u - tu) ** 2 + (d - td) ** 2 + (c - tc) ** 2
258         if dist < best_dist:
259             best_dist, best_name = dist, name
260     return best_name
261
262
263 def analyze_bytes(data: bytes) -> dict:
264     """
265     Analyze raw image bytes and return a season result dict:
266     {
267         "season", "undertone", "description",
268         "skin_hex", "axes": {undertone, depth, clarity},
269         "palette": [{"name", "hex", "role"}...],
270         "avoid": ["#hex"...],
271     }
272     Raises ValueError on undecodable input.
273     """
274     try:
275         img = Image.open(io.BytesIO(data))
276         img.load()
277     except Exception as exc:
278         raise ValueError("Could not read image") from exc
279
280     skin = _dominant_skin_rgb(img)
281     u, d, c = _axes_from_skin(skin)
282     season = _classify((u, d, c))
283     spec = _SEASONS[season]
284
285     return {
286         "season": season,
287         "undertone": spec["undertone"],
288         "description": spec["description"],
289         "skin_hex": rgb_to_hex(skin),
290         "axes": {"undertone": round(u, 3), "depth": round(d, 3), "clarity":
291 round(c, 3)},
292         "palette": [{"name": n, "hex": h, "role": role} for (n, h, role) in
293 spec["palette"]],
294         "avoid": list(spec["avoid"]),
295     }
296
297 def palette_hexes(result: dict) -> list[str]:

```

```

297     """Convenience: pull just the hex list from an analysis result."""
298     return [c["hex"] for c in result.get("palette", [])]
299
300
301 SEASON_NAMES: list[str] = list(_SEASONS.keys())
302
303
304 def palette_for_season(season: str) -> list[str]:
305     """Hex palette for a season name (case-insensitive). Empty list if
unknown."""
306     spec = _SEASONS.get(season) or next(
307         (v for k, v in _SEASONS.items() if k.lower() == (season or
"".strip().lower())),
308         None,
309     )
310     return [h for (_n, h, _r) in spec["palette"]] if spec else []

```

A.4 Backend — Selected API Routers

Authentication router `style_back/app/routers/auth.py`

```

1  import uuid
2  import bcrypt
3  from pathlib import Path
4  from fastapi import APIRouter, HTTPException, Depends, UploadFile, File,
Request
5  from sqlalchemy.orm import Session
6  from app.schemas import UserCreate, UserLogin, Token
7  from app.database import get_db
8  from app.models import UserModel
9
10 UPLOADS_DIR = Path(__file__).parent.parent.parent / "uploads"
11 UPLOADS_DIR.mkdir(exist_ok=True)
12
13 router = APIRouter(prefix="/auth", tags=["auth"])
14
15
16 def _hash(password: str) -> str:
17     return bcrypt.hashpw(password.encode(), bcrypt.gensalt()).decode()
18
19
20 def _verify(password: str, hashed: str) -> bool:
21     return bcrypt.checkpw(password.encode(), hashed.encode())
22
23
24 @router.post("/register", response_model=Token, status_code=201)
25 def register(user: UserCreate, db: Session = Depends(get_db)):
26     if db.query(UserModel).filter(UserModel.email == user.email).first():
27         raise HTTPException(status_code=400, detail="User already exists")
28     db_user = UserModel(
29         email=user.email,
30         password_hash=_hash(user.password),
31         name=user.name,
32         city=user.city,
33     )
34     db.add(db_user)
35     db.commit()
36     db.refresh(db_user)
37     return Token(access_token=f"token-{user.email}", city=user.city,
name=user.name)
38
39
40 @router.post("/login", response_model=Token)
41 def login(credentials: UserLogin, db: Session = Depends(get_db)):

```

```

42     user = db.query(UserModel).filter(UserModel.email ==
credentials.email).first()
43     if not user or not _verify(credentials.password, user.password_hash):
44         raise HTTPException(status_code=400, detail="Invalid email or
password")
45     return Token(access_token=f"token-{user.email}", city=user.city,
name=user.name)
46
47 from app.schemas import UserProfile, UserUpdate
48 from app.auth_utils import get_current_user
49
50 @router.get("/me", response_model=UserProfile)
51 def get_me(user: UserModel = Depends(get_current_user)):
52     if not user:
53         raise HTTPException(status_code=401, detail="Not authenticated")
54     return UserProfile(
55         email=user.email,
56         name=user.name,
57         city=user.city,
58         height_cm=user.height_cm,
59         weight_kg=user.weight_kg,
60         avatar_url=user.avatar_url,
61     )
62
63 @router.put("/me", response_model=Token)
64 def update_me(update: UserUpdate, db: Session = Depends(get_db), user:
UserModel = Depends(get_current_user)):
65     if not user:
66         raise HTTPException(status_code=401, detail="Not authenticated")
67
68     if update.email and update.email != user.email:
69         if db.query(UserModel).filter(UserModel.email == update.email).first():
70             raise HTTPException(status_code=400, detail="Email already taken")
71         user.email = update.email
72
73     if update.name is not None:
74         user.name = update.name
75     if update.city is not None:
76         user.city = update.city
77     if update.password:
78         user.password_hash = _hash(update.password)
79     if update.height_cm is not None:
80         user.height_cm = update.height_cm
81     if update.weight_kg is not None:
82         user.weight_kg = update.weight_kg
83
84     db.commit()
85     db.refresh(user)
86     return Token(access_token=f"token-{user.email}", city=user.city,
name=user.name)
87
88
89 @router.post("/avatar", response_model=UserProfile)
90 async def upload_avatar(
91     request: Request,
92     file: UploadFile = File(...),
93     db: Session = Depends(get_db),
94     user: UserModel = Depends(get_current_user),
95 ):
96     if not user:
97         raise HTTPException(status_code=401, detail="Not authenticated")
98     raw_bytes = await file.read()
99     ext = Path(file.filename).suffix.lower() if file.filename else ".jpg"
100     if ext not in (".jpg", ".jpeg", ".png", ".webp"):
101         ext = ".jpg"
102     filename = f"avatar_{uuid.uuid4()}_{ext}"
103     dest = UPLOADS_DIR / filename

```

```

104     dest.write_bytes(raw_bytes)
105     user.avatar_url = f"/uploads/{filename}"
106     db.commit()
107     db.refresh(user)
108     return UserProfile(
109         email=user.email,
110         name=user.name,
111         city=user.city,
112         height_cm=user.height_cm,
113         weight_kg=user.weight_kg,
114         avatar_url=user.avatar_url,
115     )

```

Colour-analysis router `style_back/app/routers/color.py`

```

1  """
2  Selfie-based color analysis + tailored recommendations (ASANUR).
3
4  Endpoints
5      POST /color/analyze          upload a selfie, run analysis, persist + return
result
6      GET  /color/result          latest saved analysis for the current user
7      GET  /color/recommendations latest season + matched closet items + matched
shopping feed
8      GET  /color/seasons         list of supported color seasons (reference)
9  """
10
11  import json
12  from typing import Optional
13
14  from fastapi import APIRouter, Depends, File, HTTPException, UploadFile
15  from sqlalchemy.orm import Session
16
17  from app.auth_utils import get_current_user_id
18  from app.color_analysis import analyze_bytes, palette_hexes, SEASON_NAMES,
palette_for_season
19  from app.color_map import season_matches, to_rgb, nearest, normalize_name,
UNIVERSAL_NEUTRALS
20  from app.database import get_db
21  from app.models import ClosetItemModel, ColorAnalysisModel, ShoppingItemModel
22  from app.schemas import (
23      ClosetItem,
24      ColorAnalysisResult,
25      ColorAxes,
26      PaletteColor,
27      ShoppingItem,
28  )
29  from app.storage import get_storage, make_key
30
31  router = APIRouter(prefix="/color", tags=["color"])
32
33  MAX_BYTES = 8 * 1024 * 1024 # 8 MB ceiling on selfie uploads
34  ALLOWED_TYPES = {"image/jpeg", "image/jpg", "image/png", "image/webp"}
35
36
37  def _to_result(row: ColorAnalysisModel) -> ColorAnalysisResult:
38      storage = get_storage()
39      palette = [PaletteColor(**c) for c in json.loads(row.palette_json or "[]")]
40      avoid = json.loads(row.avoid_json or "[]")
41      return ColorAnalysisResult(
42          id=row.id,
43          season=row.season,
44          undertone=row.undertone,
45          description=row.description,
46          skin_hex=row.skin_hex,

```



```

47         selfie_url=storage.public_url(row.selfie_key) if row.selfie_key else
None,
48         palette=palette,
49         avoid=avoid,
50         created_at=row.created_at.isoformat() if row.created_at else None,
51     )
52
53
54     def _latest_for_user(db: Session, user_id: Optional[int]) ->
Optional[ColorAnalysisModel]:
55         q = db.query(ColorAnalysisModel)
56         if user_id:
57             q = q.filter(ColorAnalysisModel.user_id == user_id)
58         return q.order_by(ColorAnalysisModel.id.desc()).first()
59
60
61     @router.post("/analyze", response_model=ColorAnalysisResult, status_code=201)
62     async def analyze(
63         file: UploadFile = File(...),
64         db: Session = Depends(get_db),
65         user_id: Optional[int] = Depends(get_current_user_id),
66     ):
67         if user_id is None:
68             raise HTTPException(status_code=401, detail="Please sign in to analyze
your colors.")
69         content_type = (file.content_type or "").lower()
70         if content_type and content_type not in ALLOWED_TYPES:
71             raise HTTPException(status_code=400, detail="Please upload a JPEG, PNG,
or WebP image.")
72
73         data = await file.read()
74         if not data:
75             raise HTTPException(status_code=400, detail="Empty file.")
76         if len(data) > MAX_BYTES:
77             raise HTTPException(status_code=413, detail="Image too large (max 8
MB).")
78
79         # Run the color science first – no point storing an unreadable image.
80         try:
81             result = analyze_bytes(data)
82         except ValueError:
83             raise HTTPException(status_code=400, detail="Could not read that image.
Try another selfie.")
84
85         # Persist the selfie via the active storage backend (local FS or S3).
86         storage = get_storage()
87         key = make_key("selfies", file.filename, content_type or "image/jpeg")
88         try:
89             storage.save(data, key, content_type or "image/jpeg")
90         except Exception as exc:
91             raise HTTPException(status_code=502, detail=f"Storage upload failed:
{exc}")
92
93         row = ColorAnalysisModel(
94             user_id=user_id,
95             selfie_key=key,
96             season=result["season"],
97             undertone=result["undertone"],
98             skin_hex=result["skin_hex"],
99             palette_json=json.dumps(result["palette"]),
100             avoid_json=json.dumps(result["avoid"]),
101             description=result["description"],
102         )
103         db.add(row)
104         db.commit()
105         db.refresh(row)
106

```

```

107     out = _to_result(row)
108     out.axes = ColorAxes(**result["axes"]) # surface the perceptual axes for
this fresh analysis
109     return out
110
111
112 @router.get("/result", response_model=ColorAnalysisResult)
113 def get_result(
114     db: Session = Depends(get_db),
115     user_id: Optional[int] = Depends(get_current_user_id),
116 ):
117     if user_id is None:
118         raise HTTPException(status_code=401, detail="Please sign in to view
your color analysis.")
119     row = _latest_for_user(db, user_id)
120     if not row:
121         raise HTTPException(status_code=404, detail="No color analysis yet.
Upload a selfie to begin.")
122     return _to_result(row)
123
124
125 @router.get("/recommendations")
126 def recommendations(
127     limit: int = 24,
128     db: Session = Depends(get_db),
129     user_id: Optional[int] = Depends(get_current_user_id),
130 ):
131     """Latest season analysis joined with wardrobe + shopping matches by
palette."""
132     if user_id is None:
133         raise HTTPException(status_code=401, detail="Please sign in to view
recommendations.")
134     row = _latest_for_user(db, user_id)
135     if not row:
136         raise HTTPException(status_code=404, detail="No color analysis yet.
Upload a selfie to begin.")
137
138     hexes = [c["hex"] for c in json.loads(row.palette_json or "[]")]
139
140     closet_q = db.query(ClosetItemModel)
141     if user_id:
142         closet_q = closet_q.filter(ClosetItemModel.user_id == user_id)
143     closet_matches = [i for i in closet_q.all() if season_matches(i.color,
hexes)]
144
145     shop_matches = [
146         i for i in db.query(ShoppingItemModel).all() if season_matches(i.color,
hexes)
147    ][:limit]
148
149     return {
150         "analysis": _to_result(row).model_dump(),
151         "closet_matches":
[ClosetItem.model_validate(_closet_dict(i)).model_dump() for i in closet_matches],
152         "shopping_matches": [ShoppingItem.model_validate(i).model_dump() for i
in shop_matches],
153     }
154
155
156 _LOOK_TITLES = [
157     "The Signature Look",
158     "Day to Evening",
159     "The Statement Edit",
160     "Off-Duty Polish",
161 ]
162
163

```

```

164 def _build_looks(items: list[ShoppingItemModel], season: str, max_looks: int =
165 3) -> list[dict]:
166     """Assemble complete outfits from palette-matching shop items, one per
167     category."""
168     by_cat: dict[str, list[ShoppingItemModel]] = {}
169     for it in items:
170         by_cat.setdefault(it.category, []).append(it)
171
172     tops = by_cat.get("Top", [])
173     bottoms = by_cat.get("Bottom", [])
174     outer = by_cat.get("Outerwear", [])
175     shoes = by_cat.get("Shoes", [])
176     acc = by_cat.get("Accessory", [])
177
178     # How many distinct looks we can reasonably build (need at least a top +
179     bottom).
180     n = min(max_looks, len(tops), len(bottoms)) if (tops and bottoms) else 0
181     looks: list[dict] = []
182     for i in range(n):
183         chosen: list[ShoppingItemModel] = [tops[i % len(tops)], bottoms[i %
184         len(bottoms)]]
185         if outer:
186             chosen.append(outer[i % len(outer)])
187         if shoes:
188             chosen.append(shoes[i % len(shoes)])
189         if acc:
190             chosen.append(acc[i % len(acc)])
191
192         total = round(sum((it.price or 0) for it in chosen), 2)
193         looks.append({
194             "title": _LOOK_TITLES[i % len(_LOOK_TITLES)],
195             "why": f"Built entirely from your {season} palette - these tones
196             flatter your complexion together.",
197             "total_price": total,
198             "items": [ShoppingItem.model_validate(it).model_dump() for it in
199             chosen],
200         })
201     return looks
202
203 @router.get("/shop")
204 def shop_recommendations(
205     db: Session = Depends(get_db),
206     user_id: Optional[int] = Depends(get_current_user_id),
207 ):
208     """Selfie-driven shopping feed: color analysis + complete looks + single
209     pieces."""
210     if user_id is None:
211         raise HTTPException(status_code=401, detail="Please sign in for
212         recommendations.")
213     row = _latest_for_user(db, user_id)
214     if not row:
215         raise HTTPException(status_code=404, detail="No color analysis yet.
216         Upload a selfie to begin.")
217
218     hexes = [c["hex"] for c in json.loads(row.palette_json or "[]")]
219     matches = [i for i in db.query(ShoppingItemModel).all() if
220     season_matches(i.color, hexes)]
221     # Surface vivid on-palette pieces first; universal neutrals fill in after.
222     matches.sort(key=lambda i: _signature_rank(i.color, hexes))
223
224     return {
225         "analysis": _to_result(row).model_dump(),
226         "looks": _build_looks(matches, row.season),
227         "singles": [ShoppingItem.model_validate(i).model_dump() for i in
228         matches],
229     }

```

```

220
221
222 # Warm/quiet neutrals that flatter broadly but shouldn't headline a "your
colors" feed.
223 _QUIET_NEUTRALS = UNIVERSAL_NEUTRALS | {
224     "beige", "tan", "taupe", "khaki", "camel", "sand", "nude", "brown",
"chocolate",
225 }
226
227
228 def _signature_rank(color: Optional[str], hexes: list[str]) -> tuple:
229     """Sort key: vivid palette hits (group 0) before quiet neutrals (group
1)."""
230     name = normalize_name(color or "")
231     is_neutral = name in _QUIET_NEUTRALS
232     rgb = to_rgb(color)
233     if rgb is None:
234         return (2, 999.0)
235     cands = [c for c in (to_rgb(h) for h in hexes) if c is not None]
236     if not cands:
237         return (1 if is_neutral else 0, 999.0)
238     _, dist = nearest(rgb, cands)
239     return (1 if is_neutral else 0, dist)
240
241
242 def _closet_dict(item: ClosetItemModel) -> dict:
243     return {
244         "id": item.id,
245         "user_id": str(item.user_id) if item.user_id else None,
246         "name": item.name,
247         "category": item.category,
248         "image_url": item.image_url,
249         "color": item.color,
250         "formality": item.formality,
251         "formality_value": item.formality_value,
252     }
253
254
255 @router.get("/seasons")
256 def list_seasons():
257     return {"seasons": SEASON_NAMES}

```

A.5 Frontend

Application root & routing `style_front/src/App.jsx`

```

1 import React, { useState, useEffect } from 'react';
2 import { Routes, Route, Navigate, useNavigate, useLocation } from 'react-
router-dom';
3
4 // Pages
5 import LandingPage from './pages/LandingPage';
6 import AuthFlow from './pages/AuthFlow';
7 import DashboardView from './pages/DashboardView';
8 import WardrobeView from './pages/WardrobeView';
9 import GeneratorView from './pages/GeneratorView';
10 import TravelView from './pages/TravelView';
11 import ProfileView from './pages/ProfileView';
12 import ShoppingView from './pages/ShoppingView';
13 import AppShell from './components/AppShell';
14 import { ClosetFilterProvider } from './context/ClosetFilterContext';
15
16 // --- PROTECTED ROUTE COMPONENT ---
17 const ProtectedRoute = ({ children, isAuthenticated }) => {

```

```

18   const location = useLocation();
19   if (!isAuthenticated) {
20     return <Navigate to="/login" state={{ from: location }} replace />;
21   }
22   return children;
23 };
24
25 const DARK_KEY = 'style-dark-mode';
26
27 const App = () => {
28   const [isAuthenticated, setIsAuthenticated] = useState(() => {
29     try { return Boolean(localStorage.getItem('auth_token')); } catch { return
false; }
30   });
31   const [activeTab, setActiveTab] = useState('wardrobe');
32   const [darkMode, setDarkMode] = useState(() => {
33     try { return localStorage.getItem(DARK_KEY) === '1'; } catch { return
false; }
34   });
35   const navigate = useNavigate();
36   const location = useLocation();
37
38   useEffect(() => {
39     document.documentElement.classList.toggle('dark', darkMode);
40     try { localStorage.setItem(DARK_KEY, darkMode ? '1' : '0'); } catch (_) {}
41   }, [darkMode]);
42
43   // Sync Sidebar with URL
44   useEffect(() => {
45     if (location.pathname.includes('dashboard')) setActiveTab('dashboard');
46     else if (location.pathname.includes('wardrobe')) setActiveTab('wardrobe');
47     else if (location.pathname.includes('generator'))
setActiveTab('generator');
48     else if (location.pathname.includes('travel')) setActiveTab('travel');
49     else if (location.pathname.includes('shopping')) setActiveTab('shopping');
50     else if (location.pathname.includes('profile')) setActiveTab('profile');
51   }, [location]);
52
53   const handleLogin = () => {
54     setIsAuthenticated(true);
55     navigate('/app/wardrobe');
56   };
57
58   const handleLogout = () => {
59     setIsAuthenticated(false);
60     try {
61       ['auth_token', 'user_city', 'user_name', 'user_avatar'].forEach(k =>
localStorage.removeItem(k));
62     } catch {}
63     navigate('/');
64   };
65
66   const handleNavClick = (id) => {
67     setActiveTab(id);
68     navigate(`/app/${id}`);
69   };
70
71   return (
72     // FIX: Removed the <button> that was here.
73     // <Routes> can ONLY contain <Route> elements.
74     <Routes>
75       {/* PUBLIC ROUTES */}
76       <Route path="/" element={<LandingPage />} />
77
78       {/* Auth Routes */}
79       <Route path="/login" element={<AuthFlow initialView="login"
onComplete={handleLogin} />} />

```

```

80     <Route path="/register" element={<AuthFlow initialView="signup"
onComplete={handleLogin} />} /> />
81
82     {/* PROTECTED APP ROUTES */}
83     <Route path="/app" element={
84         <ProtectedRoute isAuthenticated={isAuthenticated}>
85             <ClosetFilterProvider>
86                 <AppShell activeTab={activeTab} setActiveTab={handleNavClick}
onLogout={handleLogout} darkMode={darkMode} setDarkMode={setDarkMode} />
87             </ClosetFilterProvider>
88         </ProtectedRoute>
89     >
90         <Route path="dashboard" element={<DashboardView
setView={handleNavClick} />} />
91         <Route path="wardrobe" element={<WardrobeView />} />
92         <Route path="generator" element={<GeneratorView />} />
93         <Route path="travel" element={<TravelView />} />
94         <Route path="shopping" element={<ShoppingView />} />
95         <Route path="profile" element={<ProfileView onLogout={handleLogout} />} />
/>
96     <Route path="analytics" element={<DashboardView
setView={handleNavClick} />} />
97
98     {/* Default redirect when visiting /app */}
99     <Route index element={<Navigate to="wardrobe" replace />} />
100 </Route>
101
102     {/* Catch all 404 - Redirect to Home */}
103     <Route path="*" element={<Navigate to="/" replace />} />
104 </Routes>
105 );
106 };
107
108 export default App;

```

API client `style_front/src/api/client.js`

```

1  /**
2   * API client for Style backend (FastAPI).
3   * Base URL: set REACT_APP_API_URL in .env (e.g. http://localhost:8000).
4   */
5
6  const BASE = process.env.REACT_APP_API_URL || '';
7
8  function getAuthToken() {
9      if (typeof window === 'undefined') return null;
10     return window.localStorage.getItem('auth_token');
11 }
12
13 async function request(path, options = {}) {
14     const url = `${BASE}${path}`;
15     const token = getAuthToken();
16     const res = await fetch(url, {
17         headers: {
18             'Content-Type': 'application/json',
19             ...(token ? { Authorization: `Bearer ${token}` } : {}),
20             ...options.headers
21         },
22         ...options,
23     });
24     if (!res.ok) {
25         const err = new Error(res.statusText);
26         err.status = res.status;
27         try {
28             err.body = await res.json();

```

```

29     } catch {
30       err.body = await res.text();
31     }
32     throw err;
33   }
34   if (res.status === 204) return null;
35   return res.json();
36 }
37
38 const memCache = {};
39
40 async function cachedRequest(path, cacheKey) {
41   if (memCache[cacheKey]) {
42     // Background update
43     request(path).then(res => { memCache[cacheKey] = res; }).catch(() => {});
44     return Promise.resolve(memCache[cacheKey]);
45   }
46   const res = await request(path);
47   memCache[cacheKey] = res;
48   return res;
49 }
50
51 export function getCacheSync(cacheKey) {
52   return memCache[cacheKey] || null;
53 }
54
55 function clearCache() {
56   for (let key in memCache) delete memCache[key];
57 }
58
59 export const api = {
60   health() {
61     return request('/health');
62   },
63
64   register({ name, email, password, city }) {
65     return request('/auth/register', {
66       method: 'POST',
67       body: JSON.stringify({ name, email, password, city }),
68     }).then(res => {
69       if (res?.access_token) {
70         // New identity: drop any cached per-user data from a prior account
71         first.
72         localStorage.removeItem('user_avatar');
73         localStorage.setItem('auth_token', res.access_token);
74         localStorage.setItem('user_city', res.city || '');
75         localStorage.setItem('user_name', res.name || '');
76       }
77       return res;
78     });
79   },
80
81   login({ email, password }) {
82     return request('/auth/login', {
83       method: 'POST',
84       body: JSON.stringify({ email, password }),
85     }).then(res => {
86       if (res?.access_token) {
87         // New identity: drop any cached per-user data from a prior account
88         first.
89         localStorage.removeItem('user_avatar');
90         localStorage.setItem('auth_token', res.access_token);
91         localStorage.setItem('user_city', res.city || '');
92         localStorage.setItem('user_name', res.name || '');
93       }
94       return res;
95     });
96   },

```

```

94     },
95
96     getProfile() {
97         return request('/auth/me');
98     },
99
100    updateProfile(data) {
101        return request('/auth/me', {
102            method: 'PUT',
103            body: JSON.stringify(data),
104        }).then(res => {
105            if (res?.access_token) {
106                localStorage.setItem('auth_token', res.access_token);
107                localStorage.setItem('user_city', res.city || '');
108                localStorage.setItem('user_name', res.name || '');
109            }
110            return res;
111        });
112    },
113
114    getCloset({ category = null, color = null, formality = null, search = null }
= {})) {
115        const params = new URLSearchParams();
116        if (category != null && category !== '' && category !== 'all')
params.set('category', category);
117        if (color != null && color !== '') params.set('color', color);
118        if (formality != null && formality !== '') params.set('formality',
formality);
119        if (search != null && search.trim() !== '') params.set('search',
search.trim());
120        const q = params.toString() ? `?${params.toString()}` : '';
121        return cachedRequest(`/closet${q}`, `closet-${q}`);
122    },
123
124    getClosetItem(id) {
125        return cachedRequest(`/closet/${id}`, `closet-item-${id}`);
126    },
127
128    createClosetItem(item) {
129        clearCache();
130        return request('/closet', {
131            method: 'POST',
132            body: JSON.stringify(item),
133        });
134    },
135
136    uploadClosetItem(formData) {
137        clearCache();
138        const url = `${BASE}/closet/upload`;
139        const token = getAuthToken();
140        return fetch(url, {
141            method: 'POST',
142            headers: token ? { Authorization: `Bearer ${token}` } : {},
143            body: formData,
144        }).then(async (res) => {
145            if (!res.ok) {
146                const err = new Error(res.statusText);
147                err.status = res.status;
148                try { err.body = await res.json(); } catch { err.body = await
res.text(); }
149                throw err;
150            }
151            return res.json();
152        });
153    },
154
155    updateClosetItem(id, data) {

```



```

156     clearCache();
157     return request(`/closet/${id}`, {
158       method: 'PATCH',
159       body: JSON.stringify(data),
160     });
161   },
162
163   deleteClosetItem(id) {
164     clearCache();
165     return request(`/closet/${id}`, { method: 'DELETE' });
166   },
167
168   getTodayOutfit(refresh = false) {
169     return cachedRequest(`/outfits/today${refresh ? '?refresh=true' : ''}`,
`outfit-today-${refresh}`);
170   },
171
172   generateOutfit({ context, weather_temp_c, formality_preference, vibe }) {
173     // Generative AI should not be aggressively cached, but we could cache the
exact request
174     return request('/outfits/generate', {
175       method: 'POST',
176       body: JSON.stringify({
177         context: context || 'Office',
178         weather_temp_c: weather_temp_c ?? null,
179         formality_preference: formality_preference ?? null,
180         vibe: vibe ?? 50,
181       }),
182     });
183   },
184
185   saveOutfit({ context, items, explanation }) {
186     clearCache();
187     return request('/outfits/save', {
188       method: 'POST',
189       body: JSON.stringify({ context, items, explanation }),
190     });
191   },
192
193   getSavedOutfits() {
194     return cachedRequest('/outfits/saved', 'saved-outfits');
195   },
196
197   virtualTryOn(data) {
198     return request('/outfits/try-on', {
199       method: 'POST',
200       body: JSON.stringify(data),
201     });
202   },
203
204   getSuggestionsToday() {
205     const city = typeof window !== 'undefined' ?
localStorage.getItem('user_city') : null;
206     const q = city ? `?city=${encodeURIComponent(city)}` : '';
207     return cachedRequest(`/suggestions/today${q}`, `suggestions-today-${q}`);
208   },
209
210   getWardrobeGaps() {
211     return cachedRequest('/suggestions/gaps', 'wardrobe-gaps');
212   },
213
214   getCalendarEvents(date = null) {
215     const q = date ? `?date=${encodeURIComponent(date)}` : '';
216     return cachedRequest(`/calendar/events${q}`, `calendar-events-${q}`);
217   },
218
219   createCalendarEvent(event) {

```

```

220     clearCache();
221     return request('/calendar/events', {
222       method: 'POST',
223       body: JSON.stringify(event),
224     });
225   },
226
227   deleteCalendarEvent(id) {
228     clearCache();
229     return request(`/calendar/events/${id}`, { method: 'DELETE' });
230   },
231
232   /** Get weather by city name OR lat/lon coordinates */
233   getWeather({ city, lat, lon } = {}) {
234     const params = new URLSearchParams();
235     if (city) params.set('city', city);
236     if (lat != null) params.set('lat', lat);
237     if (lon != null) params.set('lon', lon);
238     return cachedRequest(`/weather/current?${params.toString()}`, `weather-${
239   {params.toString()}`);
240   },
241
242   generatePackingList(data) {
243     return request('/travel/pack', {
244       method: 'POST',
245       body: JSON.stringify(data),
246     });
247   },
248
249   getShoppingItems({ category, tags } = {}) {
250     const params = new URLSearchParams();
251     if (category && category !== 'All') params.set('category', category);
252     if (tags) params.set('tags', tags);
253     const q = params.toString() ? `?${params.toString()}` : '';
254     return request(`/shopping/items${q}`);
255   },
256
257   shoppingTryOn({ item_id, context }) {
258     return request('/shopping/try-on', {
259       method: 'POST',
260       body: JSON.stringify({ item_id, context }),
261     });
262   },
263
264   // — Color analysis (selfie-driven recommendations) —————
265   analyzeSelfie(formData) {
266     const url = `${BASE}/color/analyze`;
267     const token = getAuthToken();
268     return fetch(url, {
269       method: 'POST',
270       headers: token ? { Authorization: `Bearer ${token}` } : {},
271       body: formData,
272     }).then(async (res) => {
273       if (!res.ok) {
274         const err = new Error(res.statusText);
275         err.status = res.status;
276         try { err.body = await res.json(); } catch { err.body = await
277   res.text(); }
278         throw err;
279       }
280       return res.json();
281     });
282   },
283
284   getColorShop() {
285     return request('/color/shop');
286   },

```

```

285
286   uploadAvatar(formData) {
287     const url = `${BASE}/auth/avatar`;
288     const token = getAuthToken();
289     return fetch(url, {
290       method: 'POST',
291       headers: token ? { Authorization: `Bearer ${token}` } : {},
292       body: formData,
293     }).then(async (res) => {
294       if (!res.ok) {
295         const err = new Error(res.statusText);
296         err.status = res.status;
297         try { err.body = await res.json(); } catch { err.body = await
res.text(); }
298         throw err;
299       }
300       return res.json();
301     });
302   },
303
304   updateBodyMeasurements({ height_cm, weight_kg }) {
305     return this.updateProfile({ height_cm, weight_kg });
306   },
307 };
308
309 export const isApiConfigured = () => Boolean(BASE);

```

Tailwind theme tokens `style_front/tailwind.config.js`

```

1  /** @type {import('tailwindcss').Config} */
2  module.exports = {
3    darkMode: "class",
4    content: ["../src/**/*..{js,jsx,ts,tsx,html}"],
5    theme: {
6      extend: {
7        colors: {
8          // FitCheck AI - Luxury Fashion. Semantic tokens backed by CSS vars in
index.css.
9          // Light = ivory + gold; .dark = espresso + gold. Same classes, both
themes.
10         canvas:      "rgb(var(--c-canvas) / <alpha-value>)",    // app
background
11         card:        "rgb(var(--c-card) / <alpha-value>)",      // raised
surface / cards
12         field:       "rgb(var(--c-field) / <alpha-value>)",      // inputs /
sunken
13         beige:       "rgb(var(--c-beige) / <alpha-value>)",      // secondary
warm surface
14         line:        "rgb(var(--c-line) / <alpha-value>)",      // hairline
border
15         "line-strong": "rgb(var(--c-line-strong) / <alpha-value>)",
16         fg:          "rgb(var(--c-fg) / <alpha-value>)",        // primary
text
17         muted:       "rgb(var(--c-muted) / <alpha-value>)",      // secondary
text
18         subtle:      "rgb(var(--c-subtle) / <alpha-value>)",    //
tertiary/faint text
19         gold:        "rgb(var(--c-gold) / <alpha-value>)",      // luxury gold
accent (fills/borders)
20         "gold-soft":  "rgb(var(--c-gold-soft) / <alpha-value>)", // readable
gold for text labels
21         primary:     "rgb(var(--c-gold) / <alpha-value>)",      // alias ->
gold
22         "primary-dark": "rgb(var(--c-gold-dark) / <alpha-value>)",

```

```

23     error:          "rgb(var(--c-error) / <alpha-value>)",
24     success:        "rgb(var(--c-success) / <alpha-value>)",
25     // fixed (do not theme): dark text that always sits on gold fills
26     charcoal: "#1F2937",
27     clay: "#8C8175",
28     champagne: "#B7A98F",
29     // legacy aliases -> tokens so any stray refs stay coherent in both
themes
30     ink:             "rgb(var(--c-fg) / <alpha-value>)",
31     bone:            "rgb(var(--c-canvas) / <alpha-value>)",
32     "background-light": "rgb(var(--c-canvas) / <alpha-value>)",
33     "background-dark": "rgb(var(--c-canvas) / <alpha-value>)",
34     "card-dark":      "rgb(var(--c-card) / <alpha-value>)",
35     "surface-2":      "rgb(var(--c-field) / <alpha-value>)",
36     "sidebar-border": "rgb(var(--c-line) / <alpha-value>)",
37     "border-subtle":  "rgb(var(--c-line) / <alpha-value>)",
38     "border-medium":  "rgb(var(--c-line-strong) / <alpha-value>)",
39   },
40   fontFamily: {
41     // Poppins - workhorse UI/body. Playfair Display - luxury headings +
wordmark.
42     display: ["Poppins", "system-ui", "sans-serif"],
43     sans: ["Poppins", "system-ui", "sans-serif"],
44     serif: ["Playfair Display", "Georgia", "serif"],
45   },
46   letterSpacing: {
47     wordmark: "0.34em",
48   },
49 },
50 },
51 plugins: [],
52 }

```